

Práctica: Patrón Repository y Unit of Work en Infraestructure

Paso a paso — Desacoplar el acceso a datos y coordinar transacciones en la capa de persistencia

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 5 — Repositorios, Servicios y CRUD
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Básico (guía principal de la sesión)
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Definir el contrato genérico `IRepository<T>` en la capa Application para desacoplar el acceso a datos.
- Crear la interfaz especializada `ISolicitudRepository` con métodos para agregados y paginación en el motor relacional.
- Implementar la clase base `Repository<T>` y la clase especializada `SolicitudRepository` en Infrastructure.
- Definir e implementar el patrón Unit of Work (`IUnitOfWork` y `UnitOfWork`) para asegurar confirmaciones atómicas.
- Configurar la inyección de dependencias con ciclo de vida Scoped en la extensión `AddInfrastructureServices`.

2. Desarrollo paso a paso

Paso 1 — Definir la interfaz genérica `IRepository<T>`

En `Application/Interfaces/IRepository.cs` establecemos el contrato base que abstrae las operaciones CRUD fundamentales sobre cualquier entidad de dominio, utilizando expresiones lambda para búsquedas flexibles y tokens de cancelación.

Application/Interfaces/IRepository.cs

```
using System.Linq.Expressions;

namespace SolicitudesApp.Application.Interfaces;

public interface IRepository<T> where T : class
{
    Task<T?> GetByIdAsync(int id, CancellationToken ct = default);
    Task<IReadOnlyList<T>> ListAllAsync(CancellationToken ct = default);
    Task<IReadOnlyList<T>> FindAsync(Expression<Func<T, bool>> predicate, CancellationToken ct = default);
    Task<T> AddAsync(T entity, CancellationToken ct = default);
    Task UpdateAsync(T entity, CancellationToken ct = default);
    Task DeleteAsync(T entity, CancellationToken ct = default);
}
```

Propósito del patrón Repository: abstrae la fuente de datos subyacente para que la lógica de aplicación dependa de contratos limpios y no de DbContext directamente, permitiendo desacoplamiento estricto y facilitando pruebas unitarias con dobles de prueba (mocks).

Paso 2 — Crear el contenedor PagedResult<T> y la interfaz especializada ISolicitudRepository

Para que ISolicitudRepository pueda retornar resultados paginados sin acoplarse al motor relacional, definimos el record genérico PagedResult<T> en DTOs y luego creamos el contrato especializado ISolicitudRepository con métodos para el agregado completo y paginación en el servidor:

Archivo 1: SolicitudesApp.Application/DTOs/PagedResult.cs — Contenedor genérico para resultados de consultas paginadas:

Application/DTOs/PagedResult.cs

```
namespace SolicitudesApp.Application.DTOs;

public record PagedResult<T>(
    IReadOnlyList<T> Items,
    int TotalItems,
    int PaginaActual,
    int TamanoPagina,
    int TotalPaginas
);
```

Archivo 2: SolicitudesApp.Application/Interfaces/ISolicitudRepository.cs — Contrato especializado para el agregado Solicitud:

Application/Interfaces/ISolicitudRepository.cs

```
using SolicitudesApp.Application.DTOs;
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Application.Interfaces;

public interface ISolicitudRepository : IRepository<Solicitud>
{
    Task<Solicitud?> GetByIdConDetallesAsync(int id, CancellationToken ct = default);
    Task<PagedResult<Solicitud>> ObtenerPaginadoAsync(int pagina, int tamanoPagina, EstadoSolicitud? estado, CancellationToken ct = default);
}
```

Especialización sobre generalización: evita sobrecargar el repositorio genérico con métodos que solo aplican a una entidad concreta. ISolicitudRepository encapsula las relaciones complejas de navegación y la carga eficiente del agregado Solicitud. PagedResult<T> vive en Application/DTOs porque es un contrato de salida y no una entidad de dominio.

Paso 3 — Implementar Repository<T> y SolicitudRepository en Infrastructure

En Infrastructure/Repositorios/ implementamos las operaciones con EF Core. Repository<T> manipula el DbSet<T> genérico y SolicitudRepository aplica Include() explícitos y paginación con Skip() y Take() en SQL:

Archivo 1: SolicitudesApp.Infrastructure.Repositorios/Repository.cs — Repositorio genérico base con Entity Framework Core:

Infrastructure/Repositorios/Repository.cs

```
using System.Linq.Expressions;
using Microsoft.EntityFrameworkCore;
using SolicitudesApp.Application.Interfaces;
using SolicitudesApp.Infrastructure.Persistencia;

namespace SolicitudesApp.Infrastructure.Repositorios;

public class Repository<T> : IRepository<T> where T : class
{
    protected readonly SolicitudesDbContext _context;
    protected readonly DbSet<T> _dbSet;

    public Repository(SolicitudesDbContext context)
    {
        _context = context;
        _dbSet = context.Set<T>();
    }

    public async Task<T?> GetByIdAsync(int id, CancellationToken ct = default) =>
        await _dbSet.FindAsync(new object?[] { id }, cancellationToken: ct);

    public async Task<IReadOnlyList<T>> ListAllAsync(CancellationToken ct = default) =>
        await _dbSet.AsNoTracking().ToListAsync(ct);

    public async Task<IReadOnlyList<T>> FindAsync(Expression<Func<T, bool>> predicate,
        CancellationToken ct = default) =>
        await _dbSet.AsNoTracking().Where(predicate).ToListAsync(ct);

    public async Task<T> AddAsync(T entity, CancellationToken ct = default)
    {
        await _dbSet.AddAsync(entity, ct);
        return entity;
    }

    public Task UpdateAsync(T entity, CancellationToken ct = default)
    {
        _context.Entry(entity).State = EntityState.Modified;
        return Task.CompletedTask;
    }

    public Task DeleteAsync(T entity, CancellationToken ct = default)
    {
        _dbSet.Remove(entity);
        return Task.CompletedTask;
    }
}
```

Archivo 2: SolicitudesApp.Infrastructure/Repositorios/SolicitudRepository.cs — Repositorio especializado con carga voraz (Eager Loading) y paginación en base de datos:

Infrastructure/Repositorios/SolicitudRepository.cs

```
using Microsoft.EntityFrameworkCore;
using SolicitudesApp.Application.DTOS;
using SolicitudesApp.Application.Interfaces;
using SolicitudesApp.Domain.Entidades;
using SolicitudesApp.Infrastructure.Persistencia;

namespace SolicitudesApp.Infrastructure.Repositorios;

public class SolicitudRepository : Repository<Solicitud>, ISolicitudRepository
{
    public SolicitudRepository(SolicitudesDbContext context) : base(context) { }

    public async Task<Solicitud?> GetByIdConDetallesAsync(int id, CancellationToken ct = default)
    {
        return await _dbSet
            .Include(s => s.Detalles)
            .Include(s => s.Solicitante)
            .Include(s => s.CentroCosto)
            .Include(s => s.Aprobaciones)
            .FirstOrDefaultAsync(s => s.Id == id, ct);
    }

    public async Task<PagedResult<Solicitud>> ObtenerPaginadoAsync(int pagina, int tamanoPagina,
        EstadoSolicitud? estado, CancellationToken ct = default)
    {
        var query = _dbSet
            .Include(s => s.Detalles)
            .Include(s => s.Solicitante)
            .Include(s => s.CentroCosto)
            .AsNoTracking();

        if (estado.HasValue)
            query = query.Where(s => s.Estado == estado.Value);

        var total = await query.CountAsync(ct);
        var items = await query
            .OrderByDescending(s => s.FechaCreacion)
            .Skip((pagina - 1) * tamanoPagina)
            .Take(tamanoPagina)
            .ToListAsync(ct);

        var totalPaginas = (int)Math.Ceiling(total / (double)tamanoPagina);
        return new PagedResult<Solicitud>(items, total, pagina, tamanoPagina, totalPaginas);
    }
}
```

No llamar a SaveChanges en los repositorios: los métodos AddAsync, UpdateAsync y DeleteAsync solo configuran el estado de rastreo en el ChangeTracker de EF Core. La persistencia física se coordina de manera centralizada en el Unit of Work.

Paso 4 — Diseñar e implementar el patrón Unit of Work

Para coordinar operaciones entre múltiples repositorios asegurando transaccionalidad atómica (commit o rollback unificado), definimos el contrato IUnitOfWork en Application y su implementación concreta en Infrastructure:

Archivo 1: SolicitudesApp.Application/Interfaces/IUnitOfWork.cs — Contrato del patrón Unit of Work en la capa de negocio:

Application/Interfaces/IUnitOfWork.cs

```
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Application.Interfaces;

public interface IUnitOfWork : IDisposable
{
    ISolicitudRepository Solicitudes { get; }
    IRepository<CentroCosto> CentrosCosto { get; }
    IRepository<Usuario> Usuarios { get; }
    IRepository<Aprobacion> Aprobaciones { get; }
    Task<int> SaveChangesAsync(CancellationTokens ct = default);
}
```

Archivo 2: SolicitudesApp.Infrastructure/Repositorios/UnitOfWork.cs — Implementación del Unit of Work con instanciación diferida (lazy evaluation):

Infrastructure/Repositorios/UnitOfWork.cs

```
using SolicitudesApp.Application.Interfaces;
using SolicitudesApp.Domain.Entidades;
using SolicitudesApp.Infrastructure.Persistencia;

namespace SolicitudesApp.Infrastructure.Repositorios;

public class UnitOfWork : IUnitOfWork
{
    private readonly SolicitudesDbContext _context;
    private ISolicitudRepository? _solicitudes;
    private IRepository<CentroCosto>? _centrosCosto;
    private IRepository<Usuario>? _usuarios;
    private IRepository<Aprobacion>? _aprobaciones;

    public UnitOfWork(SolicitudesDbContext context) => _context = context;

    public ISolicitudRepository Solicitudes =>
        _solicitudes ??= new SolicitudRepository(_context);

    public IRepository<CentroCosto> CentrosCosto =>
        _centrosCosto ??= new Repository<CentroCosto>(_context);

    public IRepository<Usuario> Usuarios =>
        _usuarios ??= new Repository<Usuario>(_context);

    public IRepository<Aprobacion> Aprobaciones =>
        _aprobaciones ??= new Repository<Aprobacion>(_context);

    public async Task<int> SaveChangesAsync(CancellationTokens ct = default) =>
        await _context.SaveChangesAsync(ct);

    public void Dispose() => _context.Dispose();
}
```

Consistencia transaccional atómica: si un caso de uso modifica una Solicitud, descuenta el presupuesto de un CentroCosto y crea un registro de Aprobacion, una única llamada a SaveChangesAsync() confirma todos los cambios dentro de una sola transacción de base de datos.

Paso 5 — Registrar repositorios y Unit of Work en Inyección de Dependencias

En `Infrastructure/DependencyInjection.cs` registramos el `DbContext` con SQL Server, el Unit of Work, el repositorio especializado y el repositorio genérico con ciclo de vida `Scoped`, garantizando que compartan la misma instancia de `DbContext` durante cada petición HTTP.

`Infrastructure/DependencyInjection.cs`

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using SolicitudesApp.Application.Interfaces;
using SolicitudesApp.Application.Services;
using SolicitudesApp.Infrastructure.Persistencia;
using SolicitudesApp.Infrastructure.Repositorios;

namespace SolicitudesApp.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructureServices(this IServiceCollection services,
        string connectionString)
    {
        services.AddDbContext<SolicitudesDbContext>(options =>
            options.UseSqlServer(connectionString));

        // Registro de Unit of Work y Repositorios con ciclo Scoped
        services.AddScoped<IUnitOfWork, UnitOfWork>();
        services.AddScoped<ISolicitudRepository, SolicitudRepository>();
        services.AddScoped(typeof(IRepository<>), typeof(Repository<>));
        services.AddScoped<ISolicitudService, SolicitudService>();

        return services;
    }
}
```

Registro del genérico abierto `typeof(IRepository<>)`: al registrar `typeof(IRepository<>)`, `typeof(Repository<>)` le indicamos al contenedor de .NET que resuelva cualquier `IRepository<T>` específico (como `IRepository<Usuario>` o `IRepository<CentroCosto>`) creando una instancia de `Repository<T>`.

3. Retos adicionales

1. Agrega un método en `ISolicitudRepository` para verificar si existe alguna solicitud pendiente para un centro de costo específico.
2. Implementa un método genérico `BuscarPrimeroAsync(Expression<Func<T, bool>> predicate)` en `IRepository<T>` y `Repository<T>`.
3. Crea una prueba unitaria simulando `ISolicitudRepository` con `Moq` para verificar que el servicio no accede a la base de datos si las validaciones previas fallan.
4. Explica por qué los métodos `UpdateAsync` y `DeleteAsync` en `Repository<T>` marcan el estado en `Entry(entity)` en lugar de consultar la entidad primero a la base de datos.

4. Entregables

- Contratos IRepository<T>, ISolicitudRepository e IUnitOfWork ubicados en la capa Application.
- Implementaciones Repository<T>, SolicitudRepository y UnitOfWork en Infrastructure compilando exitosamente.
- Clase de extensión AddInfrastructureServices registrando todos los componentes con ciclo Scoped.
- Explicación técnica de cómo el Unit of Work asegura transaccionalidad atómica entre múltiples repositorios.

Técnico de Desarrollo Full Stack con C# y Angular · Sesión 5 · Docente: Edwin Barrientos · New Horizons